

PROJET BTS SIO — 1ère Année

Stack de Monitoring

Grafana + Prometheus + Node Exporter

*Mise en place d'une infrastructure de supervision système
avec Docker et Docker Compose*

Mars 2026

I. Introduction

Ce projet a été réalisé dans le cadre de la première année de BTS SIO (Services Informatiques aux Organisations). Il porte sur la mise en place d'une stack de monitoring complète à l'aide de Docker, permettant de surveiller les performances d'un système Linux en temps réel.

L'objectif principal est double :

- Apprendre les bases de Docker et Docker Compose
- Mettre en place un système de supervision système professionnel

Les trois outils utilisés sont :

- **Node Exporter** — collecte les métriques système (CPU, RAM, disque, réseau)
- **Prometheus** — récupère et stocke les métriques toutes les 15 secondes
- **Grafana** — affiche les métriques sous forme de dashboards visuels

II. Préparation de l'environnement

a. Système d'exploitation

Système utilisé : Ubuntu 24.04.3 LTS (vérifié avec la commande ci-dessous)

```
cat /etc/os-release
```

b. Installation de Docker

La vérification initiale confirme que Docker n'est pas installé par défaut :

```
docker --version
# Retourne : Command 'docker' not found
```

❑ Attention : la commande « `sudo apt-get install docker` » installe Podman (une alternative), et non Docker. Il faut passer par le script officiel.

Installation via le script officiel Docker :

```
curl -fsSL https://get.docker.com | sh
```

Ajout de l'utilisateur au groupe Docker pour éviter de taper sudo à chaque fois :

```
sudo usermod -aG docker $USER && newgrp docker
```

Vérification de l'installation avec un test simple :

```
docker run hello-world
```

Docker a alors :

1. Cherché l'image hello-world en local → introuvable

2. Téléchargé l'image depuis Docker Hub (internet)
3. Lancé un conteneur et affiché le message "Hello from Docker!"

c. Vérification finale

```
docker --version
docker compose version
```

III. Architecture de la stack

La stack repose sur trois conteneurs qui communiquent entre eux via un réseau Docker interne créé automatiquement par Docker Compose :

Conteneur	Rôle	Port exposé
Node Exporter	Collecte les métriques du système Linux (CPU, RAM, disque...)	:9100
Prometheus	Scrape Node Exporter toutes les 15s et stocke les données	:9090
Grafana	Affiche les dashboards à partir des données Prometheus	:3000

❑ Les conteneurs communiquent entre eux par leur nom (ex: « prometheus », « node-exporter ») grâce au réseau Docker interne, sans avoir besoin de connaître leurs adresses IP.

IV. Création des fichiers de configuration

a. Structure des dossiers

Commandes de création de la structure du projet :

```
mkdir monitoring-stack
cd monitoring-stack
mkdir -p prometheus grafana/provisioning/datasources
```

Arborescence finale du projet :

```
monitoring-stack/
├── docker-compose.yml
├── prometheus/
│   └── prometheus.yml
└── grafana/
    ├── provisioning/
    │   └── datasources/
    │       └── datasource.yml
```

b. docker-compose.yml

Ce fichier est le cœur du projet. Il décrit les 3 services à lancer, leurs ports, leurs volumes et leurs dépendances. Une seule commande suffit ensuite à tout démarrer.

```
version: '3.8'

services:
  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
    restart: unless-stopped

  node-exporter:
    image: prom/node-exporter:latest
    container_name: node-exporter
    ports:
      - "9100:9100"
    volumes:
      - /proc:/host/proc:ro
      - /sys:/host/sys:ro
      - /:/rootfs:ro
    command:
      - '--path.procfs=/host/proc'
      - '--path.sysfs=/host/sys'
    restart: unless-stopped

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin
    volumes:
      - grafana-data:/var/lib/grafana
      - ./grafana/provisioning:/etc/grafana/provisioning
    depends_on:
      - prometheus
    restart: unless-stopped

volumes:
  grafana-data:
```

⚠ Avertissement : la ligne « version: '3.8' » génère un warning sur les nouvelles versions de Docker ("attribute version is obsolete"). Ce n'est pas une erreur, le fichier fonctionne parfaitement. On peut supprimer cette ligne pour éviter le message.

Explication des volumes Node Exporter :

- **/proc:/host/proc:ro** — donne accès aux processus et métriques CPU/RAM de la machine hôte
- **/sys:/host/sys:ro** — donne accès aux infos matérielles (disques, réseau...)
- **/:/rootfs:ro** — donne accès à l'espace disque des partitions

Le suffixe **:ro** signifie read-only (lecture seule) — le conteneur peut lire mais pas modifier ces fichiers système.

c. prometheus/prometheus.yml

Ce fichier indique à Prometheus quelles cibles surveiller (scraping) et à quelle fréquence.

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'node-exporter'
    static_configs:
      - targets: ['node-exporter:9100']
```

d. grafana/provisioning/datasources/datasource.yml

Ce fichier configure automatiquement la connexion entre Grafana et Prometheus au démarrage, sans avoir à le faire manuellement dans l'interface.

```
apiVersion: 1

datasources:
  - name: Prometheus
    type: prometheus
    url: http://prometheus:9090
    isDefault: true
```

V. Déploiement de la stack

a. Lancement

Depuis le dossier monitoring-stack, lancer tous les conteneurs en arrière-plan :

```
docker compose up -d
```

Docker télécharge automatiquement les images si elles ne sont pas présentes localement, puis démarre les conteneurs. Résultat obtenu :

✓ Image prom/node-exporter:latest	Pulled	21.0s
✓ Image grafana/grafana:latest	Pulled	82.3s
✓ Image prom/prometheus:latest	Pulled	60.6s
✓ Network monitoring-stack_default	Created	0.3s
✓ Volume monitoring-stack_grafana-data	Created	0.0s
✓ Container prometheus	Started	9.1s
✓ Container node-exporter	Started	9.0s
✓ Container grafana	Started	2.4s

b. Vérification

Vérifier que les 3 conteneurs sont bien en cours d'exécution :

```
docker ps
```

Les 3 conteneurs doivent afficher le statut Up. Autres commandes utiles :

docker compose logs -f	# Voir les logs en temps réel
docker compose down	# Arrêter tous les conteneurs
docker compose restart	# Redémarrer la stack

VI. Accès aux interfaces

a. Prometheus — <http://localhost:9090>

L'interface Prometheus permet de vérifier l'état des cibles surveillées et d'exécuter des requêtes PromQL.

Vérification des cibles actives : Status > Target health

Les deux cibles doivent afficher le statut UP :

- node-exporter (<http://node-exporter:9100/metrics>)
- prometheus (<http://localhost:9090/metrics>)

Exemple de requête simple à tester dans l'onglet Query :

```
up
```

Cette requête retourne 1 pour chaque service actif, 0 s'il est hors ligne.

b. Grafana — <http://localhost:3000>

Identifiants de connexion par défaut :

- Login : admin
- Mot de passe : admin (à changer à la première connexion)

Connecter Prometheus comme source de données

Connections > Data sources > Add data source > Prometheus

URL à renseigner :

```
http://prometheus:9090
```

Cliquer sur Save & Test — le message doit confirmer que la connexion fonctionne.

Importer un dashboard prêt à l'emploi

Grafana propose des milliers de dashboards communautaires. Pour le monitoring Linux :

Dashboards > Import > saisir l'ID :

```
1860
```

Sélectionner la datasource Prometheus puis cliquer sur Import. Le dashboard Node Exporter Full affiche immédiatement toutes les métriques de la machine.

VII. Résultats obtenus

Le dashboard Node Exporter Full (ID 1860) affiche en temps réel :

Métrique	Description
CPU Usage	Pourcentage d'utilisation du processeur

RAM Usage	Mémoire utilisée / disponible
Disk I/O	Lecture et écriture sur le disque
Network Traffic	Trafic entrant et sortant
System Uptime	Temps de fonctionnement de la machine
Swap Usage	Utilisation de la mémoire d'échange

□ Ces métriques proviennent directement de la machine hôte (Ubuntu) grâce aux volumes /proc et /sys montés dans le conteneur Node Exporter.

VIII. Conclusion

Ce projet a permis de mettre en place une infrastructure de monitoring complète et fonctionnelle, utilisée en entreprise dans de nombreux environnements de production.

Compétences acquises :

- Comprendre le fonctionnement des conteneurs Docker
- Écrire et structurer un fichier docker-compose.yml
- Comprendre la notion de volumes, ports et réseaux Docker
- Configurer une chaîne de collecte de métriques (Node Exporter → Prometheus → Grafana)
- Utiliser l'interface Grafana pour visualiser des données système

Points d'amélioration possibles pour aller plus loin :

- Mettre en place des alertes Grafana (notification par email ou Slack si le CPU dépasse 90%)
- Sécuriser l'accès avec HTTPS et un reverse proxy (Nginx)
- Monitorer plusieurs machines en ajoutant des cibles dans prometheus.yml
- Utiliser Loki pour centraliser les logs système en complément de Prometheus